

Retrieval-Augmented Generation (RAG) - Introduction

Retrieval-Augmented Generation (RAG) is an AI architecture that combines the reasoning and language generation capabilities of Large Language Models (LLMs) with information retrieved from external knowledge sources. Traditional LLMs generate answers based only on knowledge learned during training, which can lead to outdated information and hallucinations. RAG addresses this limitation by retrieving relevant documents at query time and supplying them as context to the model before generating a response.

The main components of a RAG system are:

1. Data Source (PDFs, websites, databases, documents)
2. Document Chunking
3. Embedding Generation
4. Vector Database Storage
5. Retrieval Engine
6. Large Language Model (LLM)

Benefits of RAG include improved factual accuracy, reduced hallucinations, access to private organizational knowledge, and the ability to work with continuously updated information without retraining the model.

RAG Implementation Workflow

A typical RAG implementation follows these steps:

1. Load Documents

Documents are loaded from sources such as PDFs, Word files, websites, or databases.

2. Split Documents into Chunks

Large documents are divided into smaller chunks using techniques such as RecursiveCharacterTextSplitter.

3. Generate Embeddings

Each chunk is converted into a vector representation using an embedding model.

4. Store Embeddings in a Vector Database

The vectors are stored in databases such as Pinecone, Chroma, or Weaviate for efficient similarity search.

5. Retrieve Relevant Chunks

When a user asks a question, the query is embedded and compared against stored vectors to find the most relevant chunks.

6. Generate the Final Answer

The retrieved chunks are sent to the LLM along with the user's question. The model uses this context to produce a grounded and accurate response.

Sample Flow:

User Question → Retriever → Relevant Chunks → LLM → Answer